

# Building LLM Reasoners Assignment 1: Building a Transformer LM

Sara Vargas Martínez sv2279

February 2026

## 1 Unicode Standard

- (a) `chr(0)` returns `\x00`.
- (b) The character's string representation is different from what is printed because it represents the null character. Since Unicode maps all possible characters to encodings, we also need something to represent the absence of a character, hence the string representation `\x00` even though it has no visible textual output.
- (c) When the character occurs in text, the print statement includes the surrounding text but outputs nothing for `\x00` not even a space since it is the null character, not simply blank.

## 2 Unicode Encoding

- (a) UTF-8 allows for a more compressed representation of characters whereas UTF-16 and UTF-32 simply provide more bytes per character than necessary, making the embedding matrix larger. This matters because a key issue with the raw Unicode standard is how sparse and large the vocabulary becomes, the larger the encoding, the more we inherit that problem. The most common symbols end up with wasted zero bytes, and rare characters get unnecessarily unique encodings, so UTF-16 and UTF-32 are worse for training a tokenizer since we want the tokenizer to learn patterns in how characters are distributed.
- (b) If we try to encode and decode a string containing rare characters, the naive decoder will fail. For instance, decoding `"sará"` byte-by-byte fails because its encoding is `[115, 97, 114, 195, 161]` the character `á` is composed of 2 bytes, and a decoder that processes one byte at a time will not correctly interpret the multi-byte combination that UTF-8 uses for rarer characters.

- (c) **Example that crashes:** `[128, 100]`  
**Running:** `print(bytes([128, 100]).decode("utf-8"))`

When testing different byte combinations, I found that 127 is the last value that can be decoded as a standalone single-byte character ( $2^7$  combinations cover all single-byte encodings). Beyond 127, the byte values are reserved for specifying multi-byte sequences. For example, `á` is encoded as `[195, 161]` where `195 > 127`. The call `print(bytes([128, 100]).decode("utf-8"))` crashes because 128 is not a valid starting byte in UTF-8; after testing different values I found that 194 is the minimum valid starting byte for a multi-byte encoding.

## 3 BPE Tokenizer Training

- (a) **Training Results.** Training the BPE tokenizer on `TinyStoriesV2-GPT4-train.txt` with a vocabulary size of 10,000 yielded the following results:

The longest token, `accomplishment`, reflects that the vocabulary size was sufficiently large to merge this 15-character word, and that it appears frequently enough in the corpus for BPE to prioritize

| Metric            | Value                                  |
|-------------------|----------------------------------------|
| Training time     | 4.14 minutes                           |
| Longest token     | <code>accomplishment</code> (15 bytes) |
| Vocabulary size   | 10,000                                 |
| Number of merges  | 9,743                                  |
| Peak memory usage | 110.86 MB                              |

Table 1: BPE tokenizer training statistics on TinyStories.

it. Training completed in 4.14 minutes despite the  $\sim 1$  GB training file, owing to parallelized pre-tokenization via `multiprocessing.Pool`, use of `re.finditer` over `re.findall`, and an incremental pair-frequency dictionary that avoids recomputing frequencies from scratch each merge iteration. Peak memory remained low at  $\sim 111$  MB through corpus chunking.

- (b) **Profiling.** The merge loop dominates runtime: despite the optimized incremental frequency-update approach, the 9,743 merge iterations (each requiring a global frequency update across all affected pairs) far outweighs the cost of the parallelized pretokenization step.

## 4 BPE Tokenizer: Encoding and Decoding

- (a) **Compression Ratio.** The TinyStories tokenizer achieves a compression ratio of 4.11 bytes per token, meaning each token represents on average  $\sim 4$  bytes of raw text. This is reasonable for a 10K vocabulary BPE tokenizer.
- (b) **Throughput.** The tokenizer achieves a throughput of approximately 1.26 MB/s. At this rate, tokenizing the full 825 GB Pile dataset would require:

$$\frac{825,000 \text{ MB}}{1.26 \text{ MB/s}} \approx 655,000 \text{ s} \approx 182 \text{ hours}$$

This highlights the need for further optimization to make large-scale tokenization practical.

- (c) **Serialization Format.** The training and validation token sequences are serialized as uint16 arrays. This is appropriate because our vocabulary size of 10,000 means every token ID lies in the range  $[0, 10,000)$ , which fits comfortably within the  $2^{16} = 65,536$  values representable by a 16-bit unsigned integer, requiring no additional bits.

## 5 Transformer Accounting

### (a) Parameter Count and Memory for GPT-2 XL

GPT-2 XL has  $V=50257$ ,  $T=1024$ ,  $L=48$ ,  $d=1600$ , and we use  $d_{ff}=4d=6400$ .

Per layer, the big weight matrices are:

- Attention:  $W_Q, W_K, W_V, W_O \Rightarrow 4d^2$
- FFN:  $W_1 \in R^{d \times 4d}$  and  $W_2 \in R^{4d \times d} \Rightarrow 8d^2$
- RMSNorm scales:  $2d$  per layer

So per layer:  $12d^2 + 2d$  parameters. Outside the blocks we have token embedding ( $Vd$ ), output embedding ( $dV$ ), positional embedding ( $Td$ ), and final RMSNorm ( $d$ ).

Total parameter count:

$$N_{\text{params}} = 2Vd + Td + L(12d^2 + 2d) + d.$$

Plugging in numbers:

$$N_{\text{params}} = 2(50257)(1600) + (1024)(1600) + 48(12 \cdot 1600^2 + 2 \cdot 1600) + 1600 \approx 1.637 \times 10^9.$$

So the model has about **1.64B parameters**. In float32, just the weights are:

$$1.637 \times 10^9 \times 4 \approx 6.55 \text{ GB}.$$

**(b) Matrix Multiplies and FLOPs for a Forward Pass (GPT-2 XL)**

Use the rule  $(m \times k)(k \times n)$  costs  $2mkn$  FLOPs. With  $T=1024$ ,  $d=1600$ ,  $d_{ff}=6400$ :

Per layer:

- QKV projections:  $3 \cdot (2Td^2) = 6Td^2$
- Output projection:  $2Td^2$
- Attention scores + weighted sum:  $2T^2d + 2T^2d = 4T^2d$
- FFN:  $2Tdd_{ff} + 2Tdd_{ff}d = 4Tdd_{ff} = 16Td^2$

So per layer total:

$$F_{\text{layer}} \approx 24Td^2 + 4T^2d.$$

Numerically:

$$24Td^2 \approx 62.91\text{B}, \quad 4T^2d \approx 6.71\text{B} \Rightarrow F_{\text{layer}} \approx 69.63\text{B FLOPs}.$$

Over 48 layers:

$$48 \times 69.63\text{B} \approx 3.34 \times 10^{12} \text{ FLOPs}.$$

The final logits matmul  $(T \times d)(d \times V)$  costs:

$$F_{\text{logits}} = 2TdV \approx 1.65 \times 10^{11} \text{ FLOPs}.$$

So total forward pass:

$$F_{\text{fwd}} \approx 3.34 \times 10^{12} + 0.165 \times 10^{12} \approx \mathbf{3.51 \times 10^{12} \text{ FLOPs}}.$$

**(c) Which Parts Dominate?**

The FFN dominates because it scales like  $16Td^2$  per layer (two big matmuls). At  $T=1024$ ,  $d=1600$ , FFN is about 41.94B FLOPs per layer out of 69.63B total, i.e. roughly **60%**. The projections (QKV + output) are the next biggest chunk, and attention ( $\propto T^2d$ ) is smaller at this context length.

**(d) How FLOPs Scale Across GPT-2 Variants (fixed  $T=1024$ )**

Per layer, the main pieces are:

$$\text{QKV+O} = 8Td^2, \quad \text{FFN} = 16Td^2, \quad \text{Attention} = 4T^2d,$$

and the logits matmul is once per forward pass:  $2TdV$ .

As  $d$  increases, the  $d^2$  terms (projections + FFN) grow much faster than attention ( $\propto d$ ), so attention becomes a smaller fraction of total FLOPs at fixed  $T$ .

**(e) GPT-2 XL with  $T = 16384$**

Attention scales as  $4T^2d$ , so increasing  $T$  from 1024 to 16384 multiplies attention cost by:

$$\left(\frac{16384}{1024}\right)^2 = \times 256.$$

For XL ( $d=1600$ ), attention per layer becomes:

$$4(16384)^2(1600) \approx 1.72 \times 10^{12} \text{ FLOPs} (\approx 1.72 \text{ TFLOPs/layer}).$$

Meanwhile FFN scales linearly in  $T$ :

$$16Td^2 = 16(16384)(1600^2) \approx 6.71 \times 10^{11} \text{ FLOPs} (\approx 0.67 \text{ TFLOPs/layer}).$$

So at long context, attention becomes the main bottleneck.

## 6 Implementing SGD in PyTorch

- (a) After running the SGD code I saw the learning rate dropped much faster the first steps at  $\text{lr}=1\text{e}1$  and then became stable. when  $\text{lr}=1\text{e}2$  the updates are too large and the loss quickly explodes. Then finally at  $1\text{e}3$  the loss diverges immediately.

## 7 Resource Accounting for Training with AdamW

(All tensors are float32, so 4 bytes per stored value.)

### (a) Peak Memory Decomposition

$B=\text{batch\_size}$ ,  $T=\text{context\_length}$ ,  $d=\text{d\_model}$ ,  $L=\text{num\_layers}$ ,  $V=\text{vocab\_size}$ ,  $H=\text{num\_heads}$ , and  $d_{ff}=4d$ .

**Parameter count (in elements).**

$$N = 2Vd + Td + L(12d^2 + 2d) + d.$$

**Parameters memory.**  $M_{\text{params}} = 4N$  bytes.

**Gradients memory.**  $M_{\text{grads}} = 4N$  bytes.

**AdamW state.** AdamW stores  $m$  and  $v$  (two extra tensors):  $M_{\text{optim}} = 8N$  bytes.

So the static part is:

$$M_{\text{static}} = M_{\text{params}} + M_{\text{grads}} + M_{\text{optim}} = 16N.$$

**Activations.** Using only the components listed in the prompt, a simple (but consistent) per-layer activation tally is:

$$A_{\text{layer}} = 17BTd + 2BHT^2,$$

and outside the blocks we keep the final RMSNorm input ( $BTd$ ) and the logits/CE buffers (about  $2BTV$ ):

$$A = L(17BTd + 2BHT^2) + BTd + 2BTV.$$

Activation memory is:

$$M_{\text{act}} = 4A.$$

**Total peak memory.**

$$\boxed{M_{\text{total}} = 16N + 4A.}$$

### (b) GPT-2 XL Numbers and Max Batch Size Under 80GB

For GPT-2 XL:  $V=50257$ ,  $T=1024$ ,  $L=48$ ,  $d=1600$ ,  $H=25$ .

Static model state:

$$N \approx 1.637 \times 10^9 \quad \Rightarrow \quad M_{\text{static}} = 16N \approx 26.19 \text{ GB.}$$

Activations scale linearly with  $B$ . The per- $B$  activation size works out to:

$$M_{\text{act}} \approx 15.83 B \text{ GB.}$$

So:

$$\text{yo} \boxed{M_{\text{total}}(B) \approx 15.83 B + 26.19 \text{ GB.}}$$

To fit in 80GB:

$$15.83 B + 26.19 \leq 80 \Rightarrow B \leq 3.4 \Rightarrow \boxed{B_{\text{max}} = 3.}$$

### (c) FLOPs per AdamW Step

From the transformer accounting above, the forward FLOPs are approximately:

$$F_{\text{fwd}} \approx L(24BTd^2 + 4BT^2d) + 2BTdV.$$

The prompt says backward is  $2 \times$  forward, so one optimizer step is:

$$F_{\text{step}} \approx 3F_{\text{fwd}}.$$

(AdamW’s update itself is  $O(N)$  elementwise ops, which is small compared to the matmuls.)

### (d) Training Time on One A100 (50% MFU)

MFU is defined in tokens/sec, so batch size 1024 here means:

$$BT = 1024 \text{ tokens/step.}$$

With  $T=1024$ , that corresponds to  $B=1$ .

A100 peak fp32 is 19.5 TFLOP/s, so at 50% MFU the effective throughput is:

$$9.75 \times 10^{12} \text{ FLOP/s.}$$

For GPT-2 XL at  $B=1$ , the forward pass is about:

$$F_{\text{fwd}} \approx 3.51 \times 10^{12} \quad \Rightarrow \quad F_{\text{step}} \approx 3F_{\text{fwd}} \approx 1.05 \times 10^{13}.$$

Time per step:

$$t_{\text{step}} \approx \frac{1.05 \times 10^{13}}{9.75 \times 10^{12}} \approx 1.08 \text{ s.}$$

Over 400K steps:

$$t_{\text{total}} \approx 400,000 \times 1.08 \approx 4.32 \times 10^5 \text{ s} \approx 5.0 \text{ days.}$$

$$\boxed{\text{About 5 days on a single A100 at 50\% MFU.}}$$

## 8 Training Together

- The script for training loop lives in student and is called train.py

## 9 Generating Text

- The script for decode lives in student and is called inference.py

## 10 Varying Learning Rates

- I explored learning rates using a mix of the HPC and my laptop’s MPS, monitoring runs on Weights & Biases and stopping early when I saw divergence to save compute. I found that above  $1.5\text{e-}2$  the validation loss would diverge, and the most stable highest learning rate was  $1.5\text{e-}2$ , with which I was able to reach a validation loss below 2. However I also explored smaller learning rates which showed a more promising divergence as it was faster and lower validation loss earlier in the steps. This was  $1\text{e-}3$  and  $3\text{e-}3$ .

My greatest bottleneck in creating more cohesive results was that my `.sh` file was processing learning rates incorrectly, when I intended to pass  $2\text{e-}3$  the runs were using a learning rate 10x larger than intended. I also ran into Singularity storage quota issues which further limited how many jobs I could run.

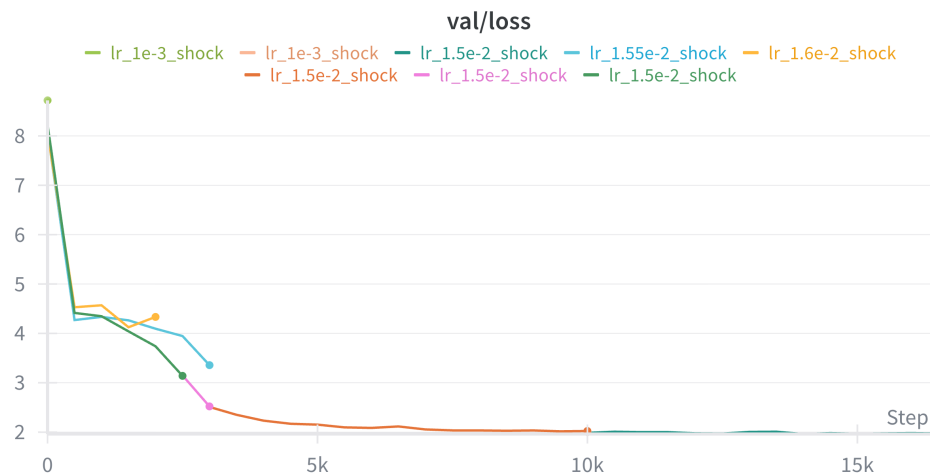


Figure 1: Here the best performing LR 1.5e-2 is left to learn and reaches a validation loss below 2

## 11 Batch Size Variation

- As expected, the higher the batch size the faster the model converges. Both batch sizes 128 and 64 converge quickly towards a validation loss around 3, with bs=128 converging slightly faster. Ideally I would have explored a wider range of batch sizes from 1 up to the GPU memory limit, but I ran into significant issues with the HPC that limited the number of jobs I could run.

## 12 Decoding

- The text generated at temperature 0.8 was somewhat incoherent, and as I decreased the temperature the output became more logical. This makes sense since a higher temperature makes the softmax distribution more uniform, increasing randomness. At temperature 0.5 the output was the most coherent. The quality of generated text is also affected by how long the model was trained and the choice of learning rate. Another inference technique is top-p sampling, where we vary the threshold that dictates how many top candidate tokens to sample from a higher threshold allows more randomness in the output.

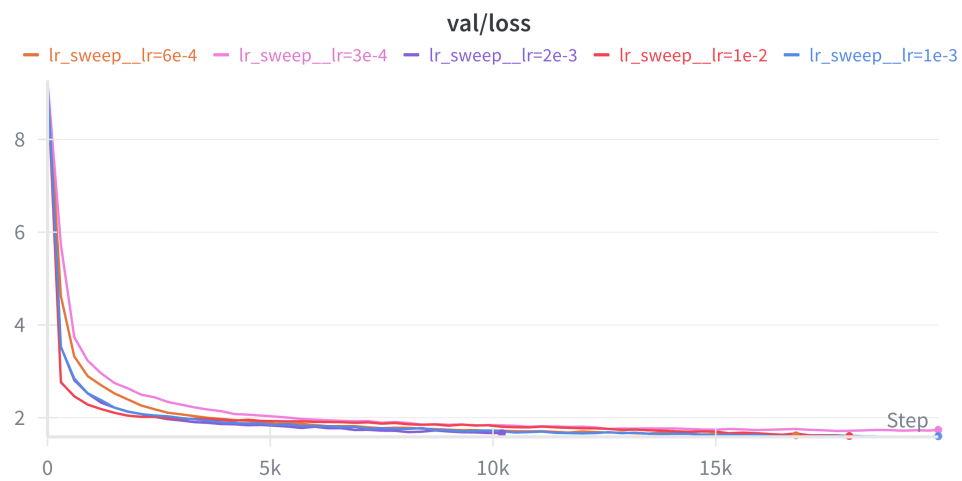


Figure 2: Earlier LR Sweep in which I tested smaller values and reached validation loss of 1.5 at 2e-3

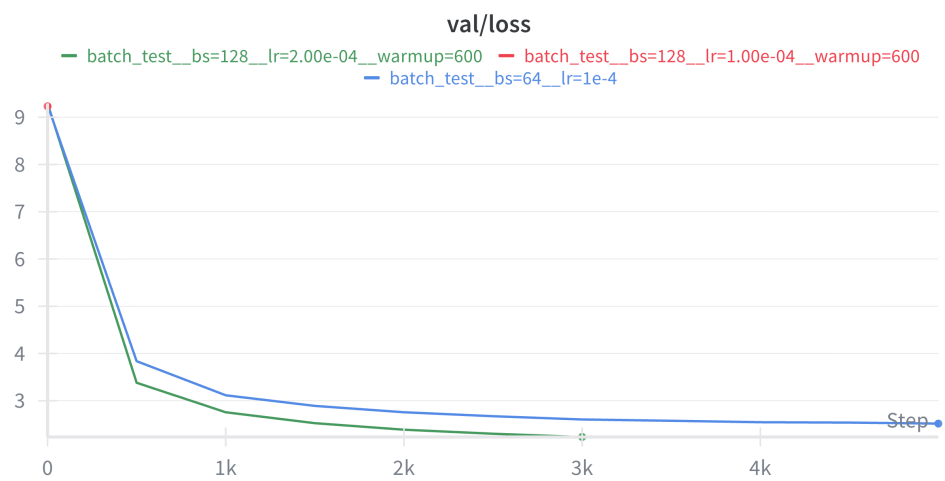


Figure 3: Varying Batch Size

Prompt: Once upon a time  
temperature=0.8

---

Once upon a time, there was an adorable little dog named Spot. Spot liked to play and play. One day, Spot saw a big ball near a tree. The ball was bouncing the ball went up high and hit the ball, near the tree. Spot was so happy! He ran the ball and bounced back goodbye. Then, Spot went to play with his new ball. They bounced high and low, and the ball bounced up high. Spot had so much fun with the ball. After playing, Spot and the ball bounced high into the sky. Spot was tired and climbed down high again. They saw the ball again, and they were happy. They played together and had a great day.

Generated 139 tokens.

Singularity> uv run inference.py --checkpoint /scratch/ql2221/checkpoints\_h200/lr\_1.5e-2\_shock/lr\_1e-02/step\_016500.pt --tokenizer\_vocab /ext3/nyu-llm-reasoners-a1/data/results/bpe\_vocab.pkl --tokenizer\_merges /ext3/nyu-llm-reasoners-a1/data/results/bpe\_merges.pkl --prompt "Once upon a time" --max\_new\_tokens 256 --temperature 1

Prompt: Once upon a time  
temperature=1.0

---

Once upon a time there were two adorable sad arms. They wanted to play together, so it would always pick up the farthest. So, the family put them on a big fight over fun moments they could make a very small tower together. At the end of the day, the family was so proud of how they built their tower. Then, the family started to make their tower even bigger! The family had done it once more. In the end, their tower was bigger filled with beautiful size and hugged each other. The family was very thankful that the competitive rabbit would be your year feel even more special than before.

Generated 123 tokens.

Singularity> uv run inference.py --checkpoint /scratch/ql2221/checkpoints\_h200/lr\_1.5e-2\_shock/lr\_1e-02/step\_016500.pt --tokenizer\_vocab /ext3/nyu-llm-reasoners-a1/data/results/bpe\_vocab.pkl --tokenizer\_merges /ext3/nyu-llm-reasoners-a1/data/results/bpe\_merges.pkl --prompt "Once upon a time" --max\_new\_tokens 256 --temperature 0.5

Prompt: Once upon a time  
temperature=0.5

---

Once upon a time, there was a little girl named Lily. She had a big, round ball that she loved to play with. One day, she found a small, shiny rock in her yard. Lily was very happy to see it. Lily's friend, Tom, came over to play. Tom saw Lily and said, "Lily, that's a special rock!" Lily was very happy and said, "Thank you, Tom! I love this rock!" They played with the rock all day long. Lily's mom was very happy to see Lily and Tom having fun. They laughed and played with the rock, and they had a great time. At the end of the day, Lily said, "Thank you, Tom! I love you!" From that day on, Lily and Tom were best friends. They played together and were best friends. And they lived happily ever after.

Generated 178 tokens.

Singularity> █

Figure 4: Testing different levels of temperature to see the different quality of text